

Mediapipe Gesture Tracking (No-PTZ)

1. Experiment Objective

MediaPipe gesture recognition is used to track the palm while recognizing the gesture category (5 fingers open / 4 fingers / 3 fingers / 2 fingers / 1 finger / fist), and the car rotates to keep the hand at the center of the frame. It supports Tracking mode (rotation only), Follow mode, and Control mode.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
Gesture Recognition	Computes the extension state of each finger based on MediaPipe hand keypoints, classifying them into 6 categories: 0–5 fingers open and fist
Fingertip Counting	Determines whether a finger is open by checking the distance relationship between the fingertip keypoints (numbered 4, 8, 12, 16, 20) and the corresponding finger roots
MediaPipe Hands	A lightweight hand keypoint detection model provided by Google that outputs the coordinates of 21 hand keypoints in real time
Gesture Control	Maps the recognized gesture category to car motion commands (e.g., 5 fingers forward, fist stop, etc.)
PD Control	Proportional-derivative controller that computes the body angular velocity based on the palm center deviation and its rate of change
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
  → gesture_tracker_node
    └─ Image processing: image conversion → flip → MediaPipe Hands → 21
        keypoints → fingersUp() classification
      └─ Body control timer (10Hz): pixel error → PD angular velocity
          (track/follow) / gesture command mapping (control)
```

Requires a MediaPipe Python environment (ROS 2 and workspace environment).

3. Core Topic Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.20~0.20 m/s	+ = forward, - = backward
			angular.z	-0.60~0.60 rad/s	+ = turn left, - = turn right

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted)

2. Start the Agent + Camera + Joint Model (Terminal 1)

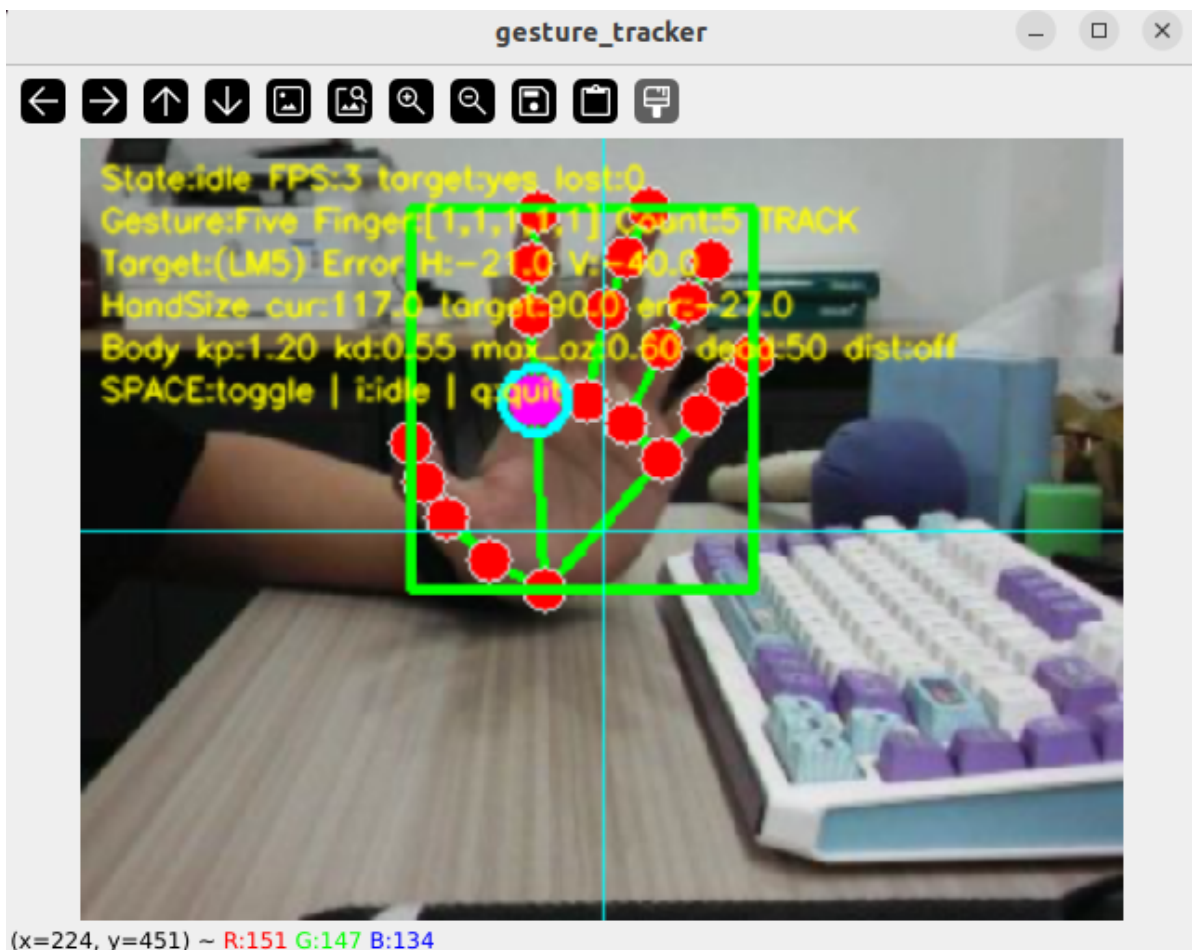
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info      | UDPv4AgentLinux.cpp | Init                               | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic...
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Gesture Tracking (Terminal 2)

Tracking mode (rotation only, showing the gesture category):

```
ros2 launch microros_v2_no_ptz_visual_control_pkg gesture_tracker.launch.py
```



4. Operation Instructions

Key	Function
Space	Toggle tracking/standby state
q / Q / ESC	Exit the program

4. Experiment Phenomenon

- When the hand moves in the frame, the car automatically rotates to track it
- The frame displays the current gesture category in real time (5 fingers open / 4 fingers / 3 fingers / 2 fingers / 1 finger / fist)
- The 21 hand keypoints and skeleton connections are visualized
- Follow mode: backs away when the hand moves closer and moves forward when it moves away
- Control mode: different gestures drive different action commands

5. Code Explanation

Source path:

- Launch (track):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/gesture_tracker.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/gesture_tracker_node.py`

- Hand detector:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/hand_detector.py`

1. Core Node Analysis

`gesture_tracker_node` adds gesture classification on top of `hand_tracker_node`. Core differences:

- **fingersUp()**: computes the extension state of the 5 fingers `[thumb, index, middle, ring, pinky]`
- **Gesture classification**: maps to 6 categories based on the number of extended fingers and specific positions
- **Control mode**: gesture categories are directly mapped to car commands (5 fingers forward, fist stop, etc.)

2. Key Parameters

Parameter definition file:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/gesture_tracker.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/gesture_tracker_node.py`

Parameter	Type	Default	Description
<code>hand_detection_confidence</code>	float	<code>0.8</code>	Hand detection confidence threshold
<code>deadband_x_pixel</code>	float	<code>50.0</code> px	Horizontal deadband
<code>angular_kp</code>	float	<code>1.2</code>	Body angle PD proportional coefficient
<code>angular_kd</code>	float	<code>0.55</code>	Body angle PD derivative coefficient
<code>max_angular_z</code>	float	<code>0.6</code> rad/s	Maximum body steering speed
<code>lost_frame_threshold</code>	int	<code>5</code>	Consecutive lost-frame tolerance threshold
<code>enable_distance_follow</code>	bool	<code>false</code>	Whether to enable distance following

3. Node Description

Node	Package	Description
<code>gesture_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	MediaPipe gesture recognition + tracking with finger-count classification

6. Common Problems

Problem	Cause	Solution
Gesture is not recognized	Insufficient lighting or the hand is too far	Improve ambient lighting and move closer to the camera
Wrong gesture classification	Insufficient fingertip detection accuracy	Keep the fingers separated and clearly visible
Control mode does not respond	Gesture recognition confidence is too low	Lower the <code>hand_detection_confidence</code> threshold